LAPORAN PRAKTIKUM

POSTTEST 6

ALGORITMA PEMROGRAMAN DASAR

Disusun oleh:

Muhammad_Rahmat_Ramadhan 2609106029

Kelas A2 ‘26

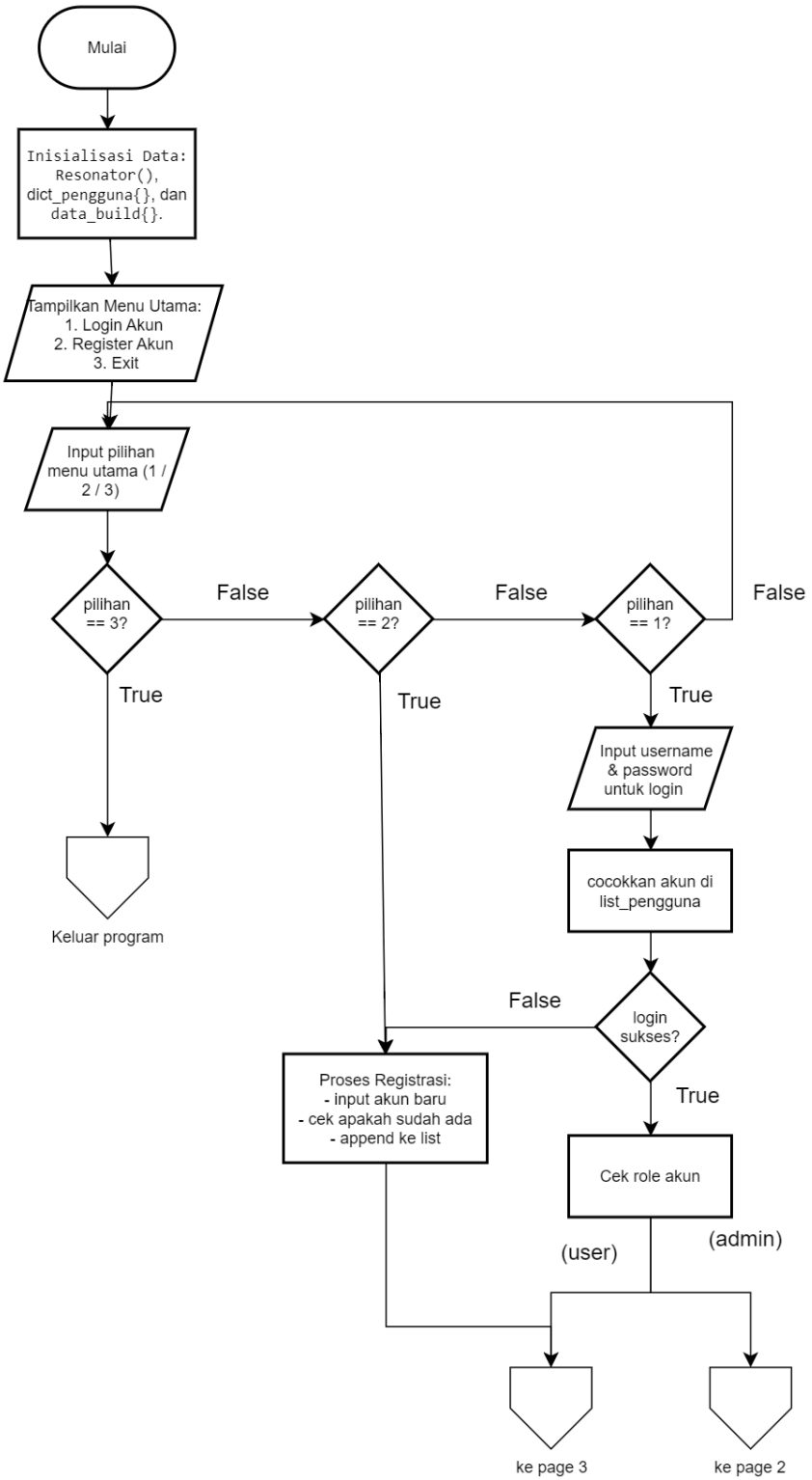
PROGRAM STUDI INFORMATIKA

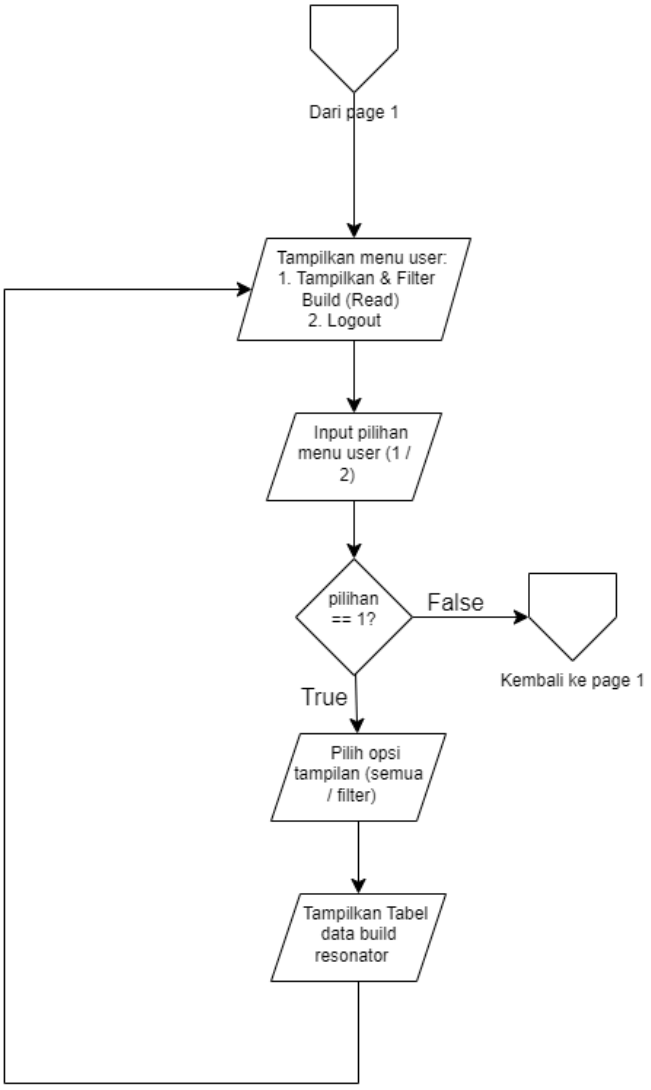
UNIVERSITAS MULAWARMAN

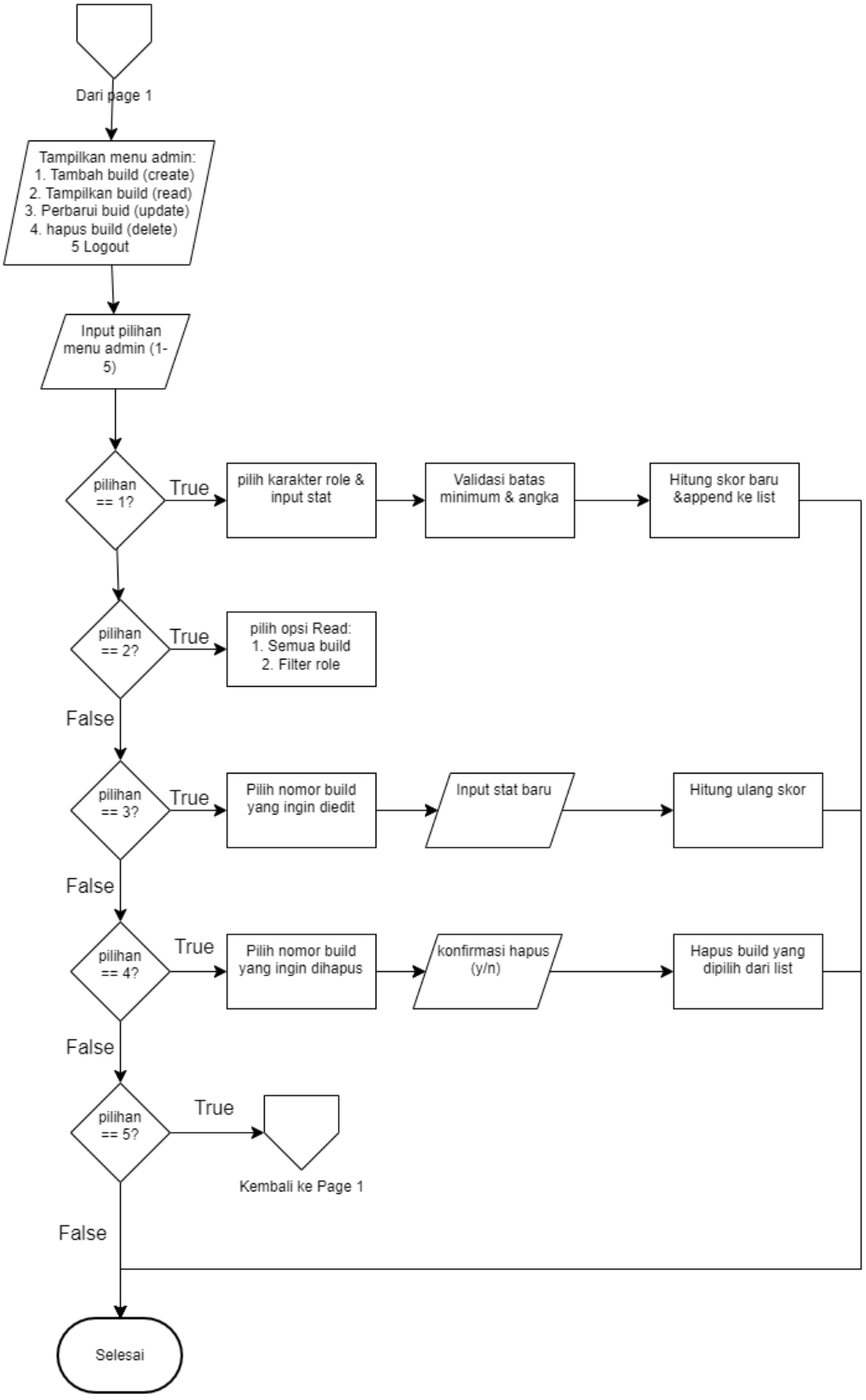
SAMARINDA

2026

1. Flowchart


Mulai
Inisialisasi Data: Resonator(), dict_pengguna{}, dan data_build{}.
Tampilkan Menu Utama:
1. Login Akun
2. Register Akun
3. Exit
Input pilihan menu utama (1 / 2 / 3)
pilihan == 3?
False
pilihan == 2?
False
pilihan == 1?
False
True
True
True
Keluar program
Input username & password untuk login
cocokkan akun di list_pengguna
False
login sukses?
True
Proses Registrasi:
- input akun baru
- cek apakah sudah ada
- append ke list
Cek role akun
(user)
(admin)
ke page 3
ke page 2


Dari page 1
Tampilkan menu user:
1. Tampilkan & Filter Build (Read)
2. Logout
Input pilihan menu user (1 / 2)
pilihan == 1?
False
Kembali ke page 1
True
Pilih opsi tampilan (semua / filter)
Tampilkan Tabel data build resonator


Dari page 1
Tampilkan menu admin:
1. Tambah build (create)
2. Tampilkan build (read)
3. Perbarui buid (update)
4. hapus build (delete)
5 Logout
Input pilihan menu admin (1-5)
pilihan == 1?
True
pilih karakter role & input stat
Validasi batas minimum & angka
Hitung skor baru &append ke list
pilihan == 2?
True
pilih opsi Read:
1. Semua build
2. Filter role
False
pilihan == 3?
True
Pilih nomor build yang ingin diedit
Input stat baru
Hitung ulang skor
False
pilihan == 4?
True
Pilih nomor build yang ingin dihapus
konfirmasi hapus (y/n)
Hapus build yang dipilih dari list
False
pilihan == 5?
True
Kembali ke Page 1
False
Selesai

Flowchart program Wuthering Waves Build & Roster Manager ini dirancang dalam 3 halaman menggunakan Off-Page Connector untuk menjaga kerapian alur sistem. Halaman 1 memuat inisialisasi struktur data berbasis Dictionary, menu utama, serta proses autentikasi (login dan registrasi pengguna). Halaman 2 menjelaskan alur untuk pengguna bertipe User biasa yang hanya memiliki akses Read-Only untuk melihat dan memfilter data build resonator. Sementara itu, Halaman 3 memuat alur kontrol penuh bagi Admin dalam mengelola data build secara menyeluruh, mencakup proses penambahan data (Create), menampilkan dan memfilter data (Read), memperbarui statistik build (Update), hingga menghapus record data (Delete) berdasarkan kunci ID unik di dictionary.

2. Deskripsi Singkat Program

Program Sistem Manajemen build Resonator di Game Wuthering Waves adalah aplikasi konsol berbasis Python yang berfungsi untuk mengelola dan mengevaluasi statistik build resonator secara efisien. Pada versi ini, sistem menggunakan struktur data Dictionary untuk mengoptimalkan penyimpanan data akun pengguna dan pencatatan build karakter dengan kunci unik. Program ini menerapkan sistem Multi-Role yang membedakan hak akses Admin (akses penuh CRUD) dan User biasa (Read-Only). Selain itu, program dilengkapi dengan validasi input angka ketat tanpa try-except, penilaian skor build otomatis, pembersihan terminal (import os), serta format tabel yang rapi tanpa menggunakan fungsi kustom (def) sesuai aturan praktikum.

3. Source Code

A. Fitur Inisialisasi Data Pengguna & menu Utama

Bagian ini mendeklarasikan master data menggunakan struktur Dictionary untuk penyimpanan akun pengguna (dict_pengguna) dan data build (data_build). Dilengkapi perulangan while utama yang mengatur jalannya navigasi menu, login, dan registrasi.

```
# Data pengguna menggunakan Dictionary
dict_pengguna = {
    "rahmat": {"password": "rahmat029", "role": "Admin"},
    "mat": {"password": "mat029", "role": "User"}
}

# Data build resonator menggunakan Dictionary
data_build = {
    1: {"nama": "Hsin", "role": "DPS", "level": 90, "hp": 20000, "atk": 2500,
"def": 1100, "crit_rate": 70.0, "crit_dmg": 255.0, "energy_regen": 130.0,
"skor": 650.0, "penilaian": "[GOD Build] Build impian yang sangat sempurna!"},
id_counter = 2

# tampilan menu utama
menu_utama = True

while menu_utama:
    os.system("cls" if os.name == "nt" else "clear")
```

```
    print("1. Login Akun")
    print("2. Registrasi Akun")
    print("3. Exit")
    print("=" * 60)
    pilihan = input("Masukkan pilihan (1/2/3): ")

    while pilihan not in ("1", "2", "3"):
        print("Pilihan tidak valid. Silakan masukkan angka 1, 2, atau 3.")
        pilihan = input("Masukkan pilihan (1/2/3): ")
```

B. Fitur Create

Fitur ini menangani proses penambahan data build baru oleh Admin. Di dalamnya terdapat perulangan validasi input angka dan perhitungan skor otomatis sebelum data disimpan ke dictionary data_build menggunakan kunci ID unik id_counter.

```
if menu_admin_pilihan == "1":
                        os.system("cls" if os.name == "nt" else "clear")

                        for urutan in range(len(Resonator)):
                            no_tampilan = urutan + 1
                            nama_resonator = Resonator[urutan]
                            print(f"{no_tampilan}. {nama_resonator}")

                        pilihan_char = input("\nPilih nomor karakter: ")

                        while not (1 <= int(pilihan_char) <= len(Resonator)):
                            print("[ERROR] Nomor tidak ada di dalam daftar!")
                            pilihan_char = input("Masukan nomor yang benar: ")

                        posisi_char = int(pilihan_char) - 1
                        nama_char = Resonator[posisi_char]


                        # Input Statistik angka
                        input_lvl = input("Level (1-90)         : ")
                        level_valid = False
                        while not level_valid:
                            if input_lvl == "":
                                input_lvl = input("Level tidak boleh kosong
(1-90): ")
                                continue
                            cek_lvl = True
                            for angka in input_lvl:
                                if angka not in ("0", "1", "2", "3", "4", "5",
"6", "7", "8", "9"):
                                    cek_lvl = False
                            if not cek_lvl:
```

```
                                input_lvl = input("[ERROR] Harus angka! Masukkan
level (1-90): ")
                            elif not (1 <= int(input_lvl) <= 90):
                                input_lvl = input("[ERROR] Level harus antara 1
sampai 90: ")
                            else:
                                level_valid = True
                        level = int(input_lvl)
# Validasi HP
                        input_hp = input("Total HP          : ")
                        hp_valid = False
                        while not hp_valid:
                            if input_hp == "":
                                input_hp = input("HP tidak boleh kosong: ")
                                continue
                            cek_hp = True
                            for angka in input_hp:
                                if angka not in ("0", "1", "2", "3", "4", "5",
"6", "7", "8", "9"):
                                    cek_hp = False
                            if not cek_hp:
                                input_hp = input("[ERROR] HP harus berupa angka
positif!: ")
                            elif int(input_hp) < 8000:
                                input_hp = input("[ERROR] HP minimal 8000!: ")
                            else:
                                hp_valid = True
                        hp = int(input_hp)

                        # Validasi input nilai ATK
                        input_atk = input("Total ATK           : ")
                        atk_valid = False
                        while not atk_valid:
                            if input_atk == "":
                                input_atk = input("ATK tidak boleh kosong: ")
                                continue
                            cek_atk = True
                            for angka in input_atk:
                                if angka not in ("0", "1", "2", "3", "4", "5",
"6", "7", "8", "9"):
                                    cek_atk = False
                            if not cek_atk:
                                input_atk = input("[ERROR] ATK harus bernilai
valid!: ")
                            elif int(input_atk) < 1200:
                                input_atk = input("[ERROR] ATK minimal 1200!: ")
                            else:
                                atk_valid = True
                        atk = int(input_atk)

                        # Validasi input nilai DEF
```

```
                        input_def = input("Total DEF            : ")
                        def_valid = False
                        while not def_valid:
                            if input_def == "":
                                input_def = input("DEF tidak boleh kosong: ")
                                continue
                            cek_def = True
                            for angka in input_def:
                                if angka not in ("0", "1", "2", "3", "4", "5",
"6", "7", "8", "9"):
                                    cek_def = False
                            if not cek_def:
                                input_def = input("[ERROR] DEF harus bernilai
valid!: ")
                            elif int(input_def) < 800:
                                input_def = input("[ERROR] DEF minimal 800!: ")
                            else:
                                def_valid = True
                        Def = int(input_def)

                        crit_rate = float(input("Total Crit Rate (%)   : "))
                        crit_dmg = float(input("Total Crit DMG (%)    : "))
                        energy_regen = float(input("Total Energy Regen    : "))

                        # Perhitungan Build
                        crit_value = (crit_rate * 2) + crit_dmg
                        skor_build = crit_value + (atk / 20) + (Def / 30) +
(energy_regen * 0.5)

                        if skor_build > 600:
                            penilaian = "[GOD Build] Build impian yang sangat
sempurna!"
                        elif skor_build >= 425:
                            penilaian = "[Epic Build] Build sudah sangat layak
pakai!"
                        elif skor_build >= 300:
                            penilaian = "[Mid Build] Build standar, masih bisa
ditingkatkan."
                        else:
                            penilaian = "[Bad Build] Build kurang, perlu farm
echo ulang!"

                        # Menyimpan ke dictionary data_build
                        data_build[id_counter] = {
                            "nama": nama_char,
                            "role": role_char,
                            "level": level,
                            "hp": hp,
                            "atk": atk,
                            "def": Def,
                            "crit_rate": crit_rate,
```

```
                            "crit_dmg": crit_dmg,
                            "energy_regen": energy_regen,
                            "skor": skor_build,
                            "penilaian": penilaian
                        }
                        id_counter += 1
```

C. Fitur Read

Fitur ini menggunakan metode .items() untuk mengakses kunci ID dan nilai dictionary secara bersamaan. Dilengkapi percabangan untuk memilih opsi tampil seluruh data atau filter berdasarkan Role, serta pemformatan f-string agar tabel sejajar rapi.

```
elif menu_admin_pilihan == "2":
                        os.system("cls" if os.name == "nt" else "clear")
                            print("1. Tampilkan Semua Build")
                            print("2. Filter Build Berdasarkan Role")
                            sub_menu_tampilan = input("Masukkan pilihan (1/2):
")

                            if sub_menu_tampilan == "1":
                                print(f"{'ID.':<5}{'Nama
Resonator':<20}{'Role':<15}{'Level':<10}{'HP':<10}{'ATK':<10}{'DEF':<10}{'Crit
Rate (%)':<15}{'Crit DMG (%)':<15}{'Energy Regen':<15}")
                                for id_build, detail in data_build.items():

print(f"{id_build:<5}{detail['nama']:<20}{detail['role']:<15}{detail['level']:<1
0}{detail['hp']:<10}{detail['atk']:<10}{detail['def']:<10}{detail['crit_rate']:<
15}{detail['crit_dmg']:<15}{detail['energy_regen']:<15}")

                            elif sub_menu_tampilan == "2":
                                filter_role = input("Masukkan role yang ingin
difilter (DPS/Sub-DPS/Support): ")
                                print(f"{'ID.':<5}{'Nama
Resonator':<20}{'Role':<15}{'Level':<10}{'HP':<10}{'ATK':<10}{'DEF':<10}{'Crit
Rate (%)':<15}{'Crit DMG (%)':<15}{'Energy Regen':<15}")
                                found = False

                                for id_build, detail in data_build.items():
                                    if detail['role'].lower() ==
filter_role.lower():

print(f"{id_build:<5}{detail['nama']:<20}{detail['role']:<15}{detail['level']:<1
0}{detail['hp']:<10}{detail['atk']:<10}{detail['def']:<10}{detail['crit_rate']:<
15}{detail['crit_dmg']:<15}{detail['energy_regen']:<15}")
                                        found = True
```

D. Fitur Update

Fitur ini memungkinkan Admin untuk memperbarui statistik build yang sudah tersimpan di dalam dictionary data_build. Program terlebih dahulu menampilkan daftar build beserta nomor ID kunci uniknya. Setelah ID target dipilih dan divalidasi, sistem akan menampilkan data lama dan menerima input statistik baru (jika pengguna menekan Enter tanpa mengisi, nilai lama tetap dipertahankan). Setelah nilai diperbarui, sistem secara otomatis menghitung ulang Crit Value, skor build, serta penilain baru untuk karakter tersebut.

```
elif menu_admin_pilihan == "3":
                        os.system("cls" if os.name == "nt" else "clear")
                                                for id_build, detail in
data_build.items():
                                print(f"{id_build}. {detail['nama']}
({detail['role']})")
                            pilihan_update = input("Pilih nomor ID build yang
ingin diperbarui: ")

                            valid_update = False
                            while not valid_update:
                                cek_update = True

                                        cek_update = False
                                if not cek_update or pilihan_update == "":
                                    pilihan_update = input("[ERROR] Pilihan
harus berupa angka valid!: ")
                                elif int(pilihan_update) not in data_build:
                                    pilihan_update = input("[ERROR] ID build
tidak ditemukan di daftar!: ")
                                else:
                                    valid_update = True

                            id_target = int(pilihan_update)
                            target = data_build[id_target]

                            print(f"\n===== Update Build untuk {target['nama']}
=====")
                            print("Tekan Enter langsung jika tidak ingin
mengubah stat tertentu.")

                            new_level = input(f"Level ({target['level']}): ")
                            if new_level != "":
                                target['level'] = int(new_level)

                            new_hp = input(f"Total HP ({target['hp']}): ")
                            if new_hp != "":
                                target['hp'] = int(new_hp)

                            new_atk = input(f"Total ATK ({target['atk']}): ")
                            if new_atk != "":
                                target['atk'] = int(new_atk)

                            new_def = input(f"Total DEF ({target['def']}): ")
                            if new_def != "":
                                target['def'] = int(new_def)

                            new_cr = input(f"Crit Rate (%)
({target['crit_rate']}%): ")
                            if new_cr != "":
                                target['crit_rate'] = float(new_cr)
```

```
                            new_cdm = input(f"Crit DMG (%)
({target['crit_dmg']}%): ")
                            if new_cdm != "":
                                target['crit_dmg'] = float(new_cdm)

                            new_er = input(f"Energy Regen
({target['energy_regen']}%): ")
                            if new_er != "":
                                target['energy_regen'] = float(new_er)

                            # Perhitungan ulang skor build setelah update
                            crit_value = (target['crit_rate'] * 2) +
target['crit_dmg']
                            skor_baru = crit_value + (target['atk'] / 20) +
(target['def'] / 30) + (target['energy_regen'] * 0.5)

                            if skor_baru > 600:
                                penilaian_baru = "[GOD Build] Build impian yang
sangat sempurna!"
                            elif skor_baru >= 425:
                                penilaian_baru = "[Epic Build] Build sudah
sangat layak pakai!"
                            elif skor_baru >= 300:
                                penilaian_baru = "[Mid Build] Build standar,
masih bisa ditingkatkan."
                            else:
                                penilaian_baru = "[Bad Build] Build kurang,
perlu farm echo ulang!"

                            target['skor'] = skor_baru
                            target['penilaian'] = penilaian_baru
```

E. Fitur Delete

Fitur ini memungkinkan Admin untuk memperbarui statistik build yang sudah tersimpan di dalam dictionary data_build. Program terlebih dahulu menampilkan daftar build beserta nomor ID kunci uniknya. Setelah ID target dipilih dan divalidasi, sistem akan menampilkan data lama dan menerima input statistik baru (jika pengguna menekan Enter tanpa mengisi, nilai lama tetap dipertahankan). Setelah nilai diperbarui, sistem secara otomatis menghitung ulang Crit Value, skor build, serta penilain baru untuk karakter tersebut.

```
 elif menu_admin_pilihan == "4":
                        os.system("cls" if os.name == "nt" else "clear")

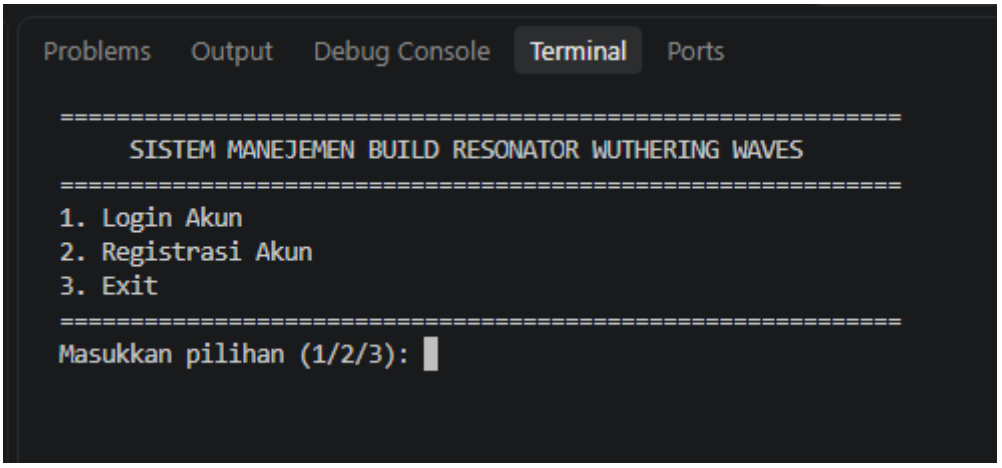
                            for id_build, detail in data_build.items():
                                print(f"{id_build}. {detail['nama']}
({detail['role']})")
                            pilihan_hapus = input("Pilih nomor ID build yang
ingin dihapus: ")

                            valid_hapus = False
                            while not valid_hapus:
                                cek_hapus = True
                                for angka in pilihan_hapus:
                                    if angka not in ("0", "1", "2", "3", "4",
"5", "6", "7", "8", "9"):
                                        cek_hapus = False
                                if not cek_hapus or pilihan_hapus == "":
                                    pilihan_hapus = input("[ERROR] Pilihan harus
berupa angka valid!: ")
                                elif int(pilihan_hapus) not in data_build:
                                    pilihan_hapus = input("[ERROR] ID build
tidak ditemukan di daftar!: ")
                                else:
                                    valid_hapus = True

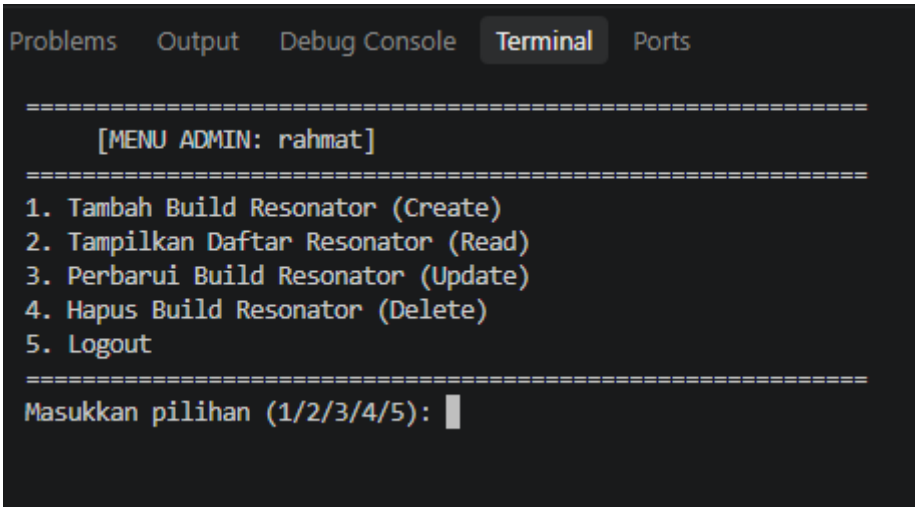
                            id_target_hapus = int(pilihan_hapus)
                            nama_hapus = data_build[id_target_hapus]['nama']

                            konfirmasi = input(f"Apakah Anda yakin ingin
menghapus build untuk {nama_hapus}? (y/n): ")
                            if konfirmasi.lower() == 'y':
                                del data_build[id_target_hapus]
                                print(f"[BERHASIL] Build untuk {nama_hapus}
berhasil dihapus.")
                            else:
                                print("[INFO] Penghapusan dibatalkan.")
                        input("Tekan Enter untuk kembali ke menu admin...")
```

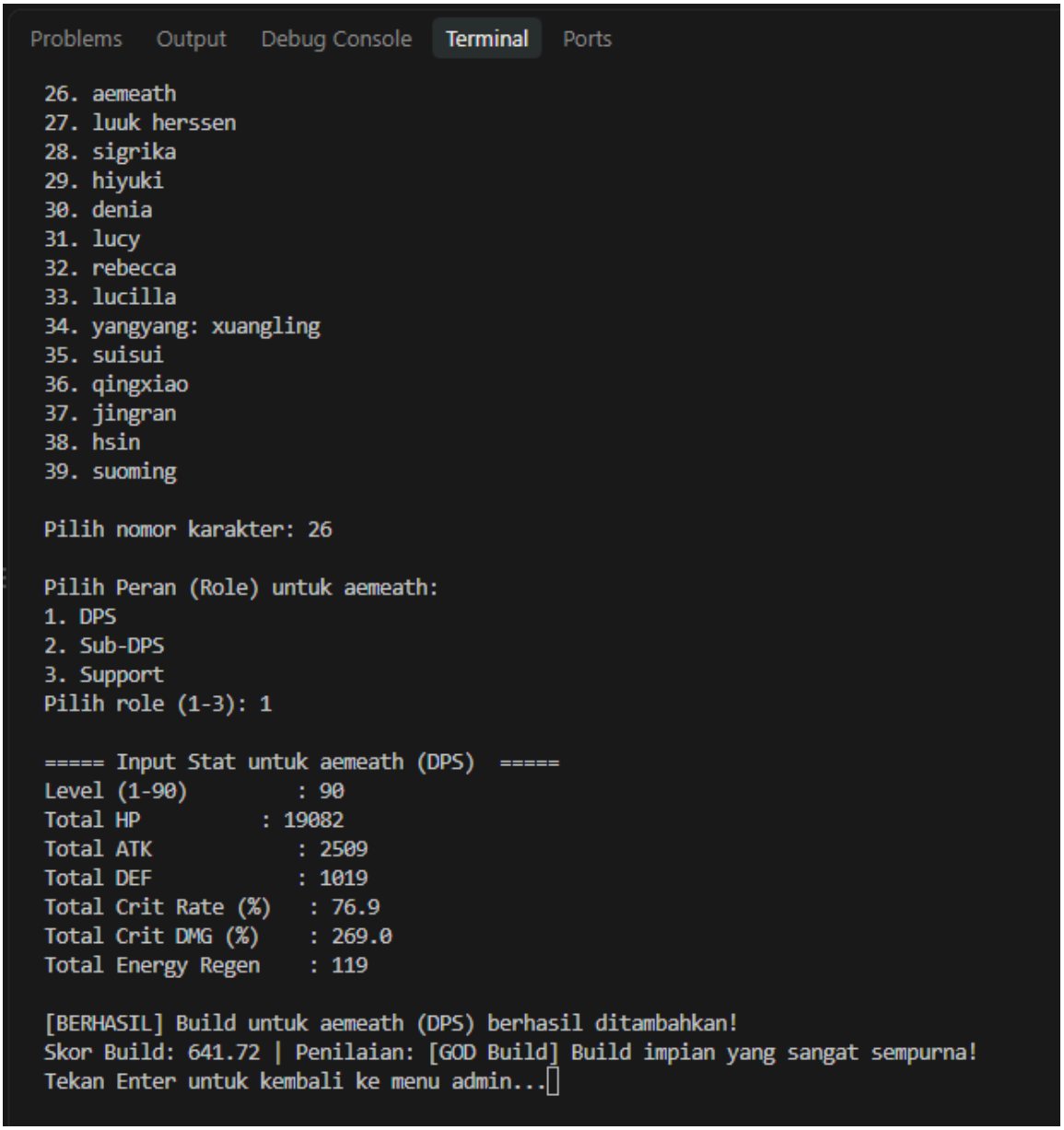
4. Hasil Output



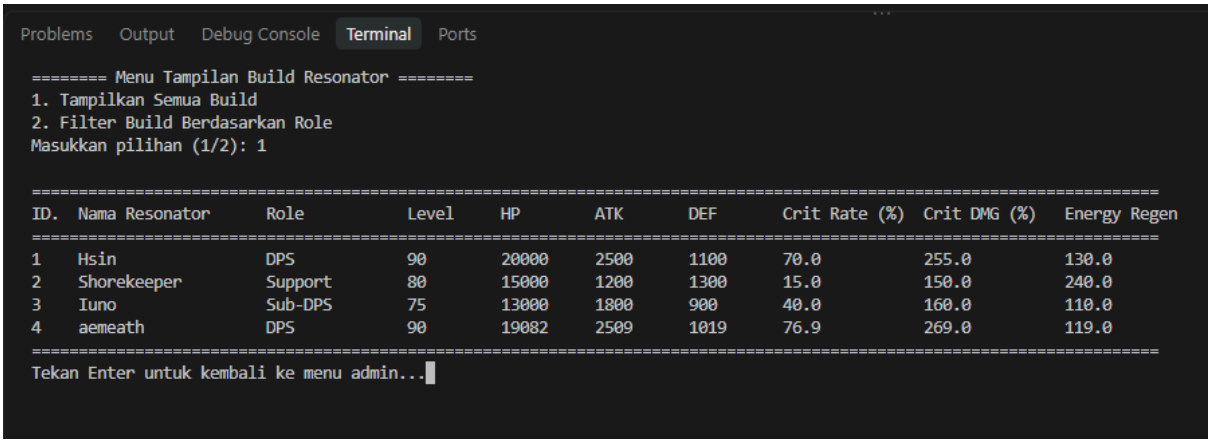
Gambar 4.1 Tampilan Menu Utama



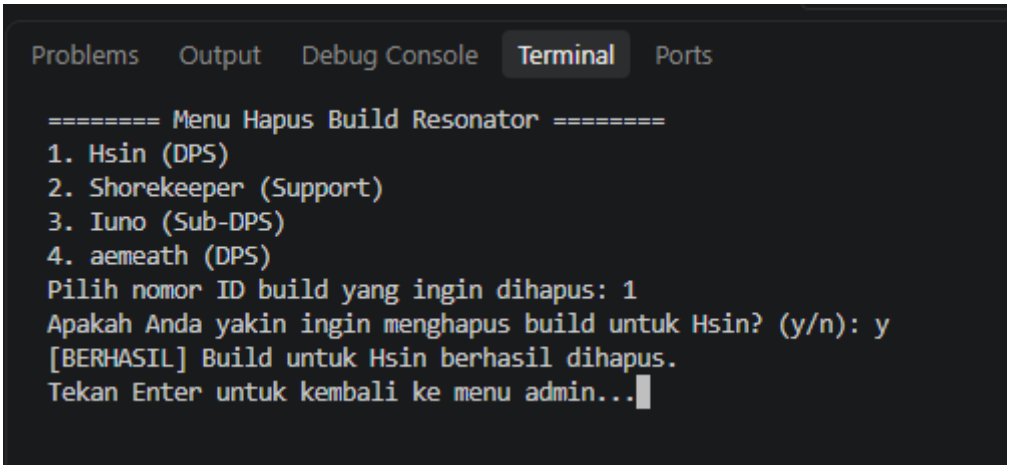
Gambar 4.2 Tampilan Menu Admin



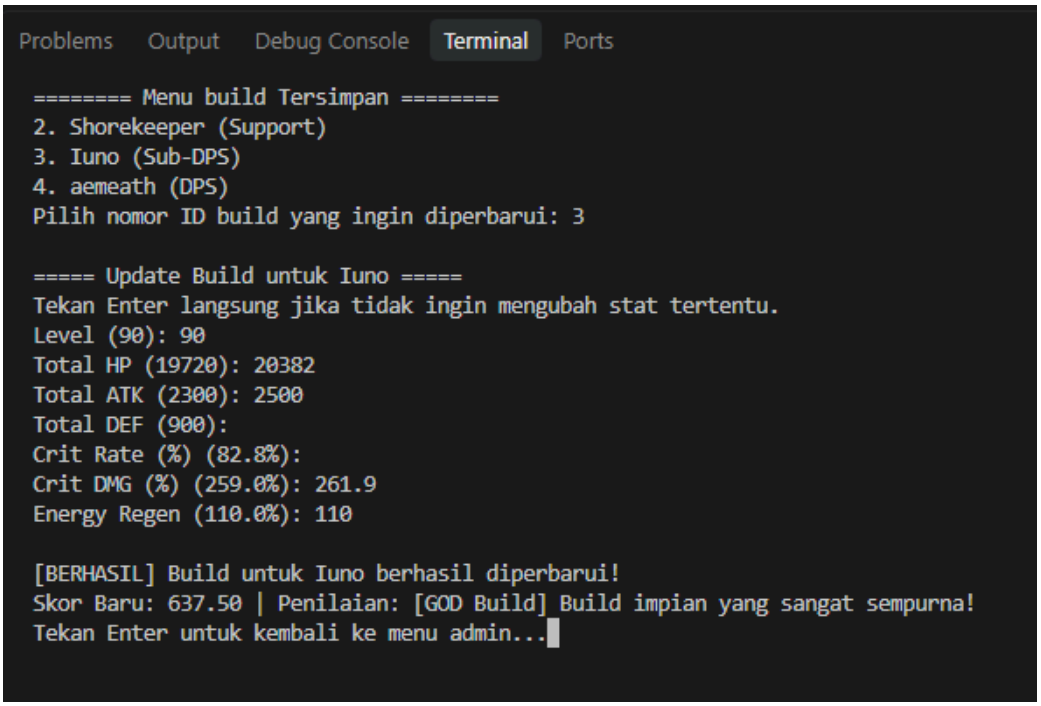
Gambar 4.3 Tampilan Fitur Create



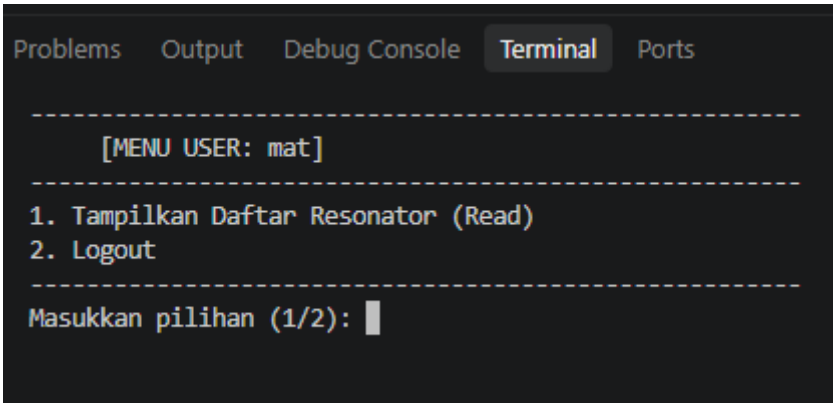
Gambar 4.4 Tampilan Fitur Read



Gambar 4.5 Tampilan Fitur Delete



Gambar 4.6 Tampilan Fitur Update

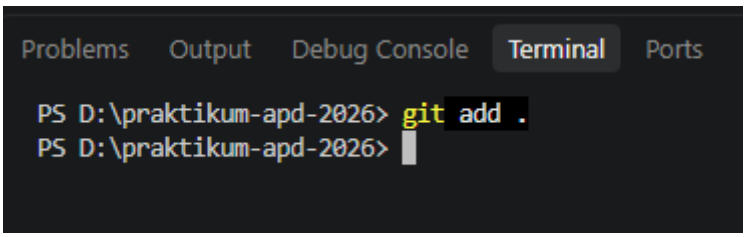


Gambar 4.7 Tampilan Menu User

5. Langkah-langkah Git

5. 1 Git Add

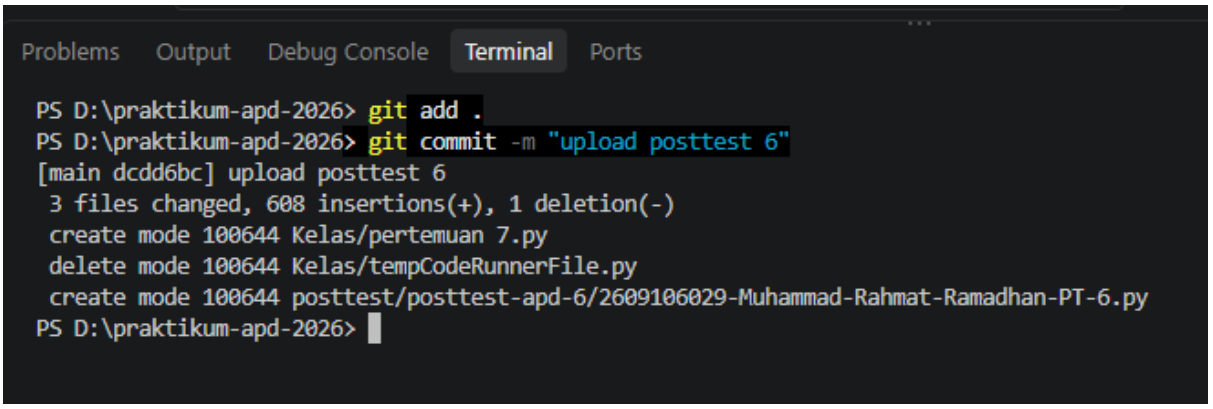
Perintah git add digunakan untuk memindahkan file kode program yang baru dibuat. Langkah ini mempersiapkan file agar siap untuk dicatat (commit).



Gambar 5.1 Perintah git add .

5.2 Git Commit

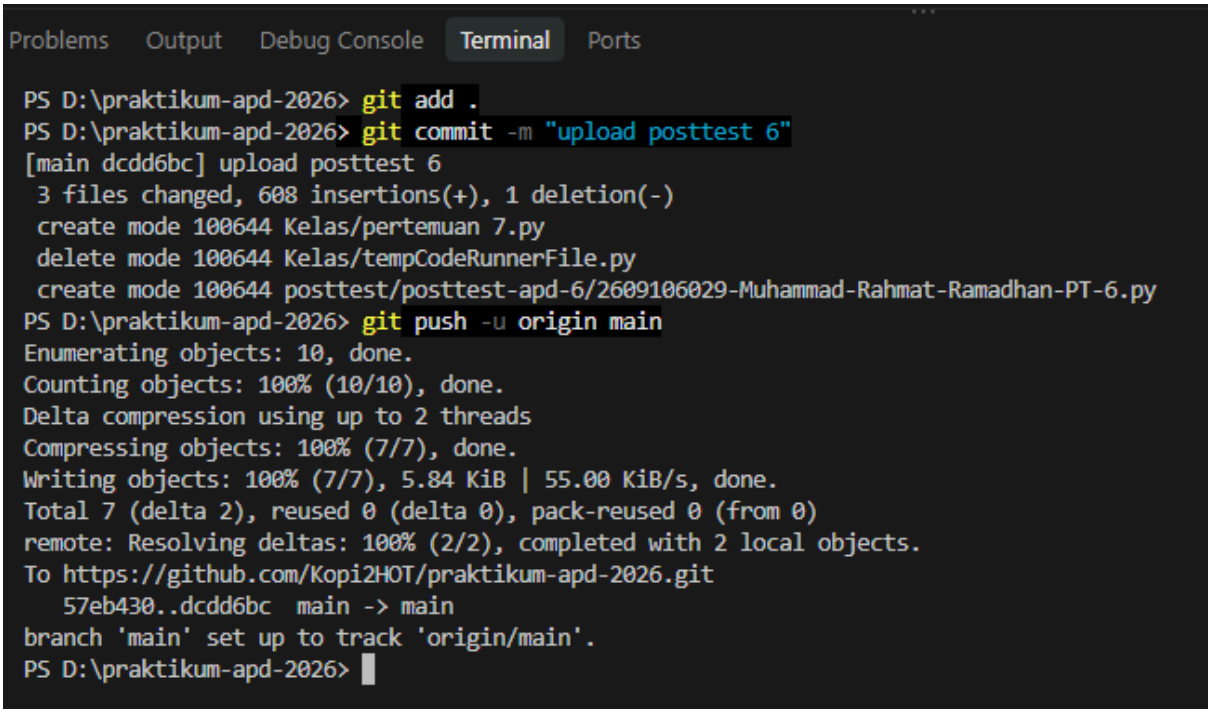
Perintah git commit berfungsi untuk menyimpan perubahan yang ada di staging area ke dalam repository lokal secara permanen. Setiap commit wajib disertai pesan deskriptif (-m) guna memperjelas riwayat perubahan apa saja yang dilakukan pada program.



Gambar 5.2 Perintah git commit -m "..."

5.3 Git Push

Perintah git push digunakan untuk mengunggah (upload) seluruh riwayat commit dari repository lokal di komputer ke repository jarak jauh (remote repository) di GitHub. Hal ini memastikan kode program tersimpan secara online.



Gambar 5.3 Perintah git push -u origin main